

```
parallel.initialize(int n, int m);
```

where n is the size of the first dimension of the process grid and m of the second, a $n * m$ geometry. The initialization of the parallel object should be the first execution of any package which use LATfield2 as all its classes depend on this object and more important as MPI is initialized within the parallel object initialization.

One should notice that a geometry of $n * m$ is not equal to $m * n$. Indeed $n * m$ means that the first dimension has size n and the second size m , so $m * n$ mean the inverse. Scalability and execution time depend on the geometry. Usually, it is most efficient to have a close to square geometry. In many case a square geometry of the process grid is not possible (for example, due to node allocation schemes on the cluster or to the problem sizes). In that case, it is always better to use a $n * m$ geometry with $n < m$. This is especially true when using the fast Fourier transform functionality.

3.1.1. I/O server. The LATfield2 parallelization allows the usage of additional MPI processes reserved for writing to disks. The server has been developed to allow the user to write relatively small amounts of data very fast, where “small” means 5-10% of the total physical memory on a node. Examples would be the positions of zeros or the local maxima of a field. The server has been implemented to maximize its performance, regardless of the structure of output files. The server writes data into several files sharing a root filename. The server processes are grouped together, reflecting typical cluster architecture of multicore nodes. The number of file parts is defined by the number of server process groups. The I/O server currently has no capability for input.

The files have no structure at all, and are composed of the list of messages sent by the compute processes. This simplicity minimizes the amount of computation done by the server, in the goal to maximize the time the server is ready to receive messages from the compute nodes. Therefore the output files will need some post production treatment to recover the correct data structure.

Figure 3.1 describes the geometry and the interconnection of the processes. Each server process is connected to several compute processes, and then combined into a group. Each server process group writes a single file using MPI parallel I/O. The server process group is best chosen to reside on a single node.

To use LATfield2 with the I/O server the flag `-DEXTERNAL_IO` must be set at compilation, and the initialization of the parallel object is done with the following method:

```
parallel.initialize(int n,int m,int io_size,int io_group_size);
```

where n and m have the same significance as before. The variable `io_size` is the number of process allocated to the I/O server, which must be an integer divisor of n , and $n/\text{io_size}$ should be an integer divisor of m . `io_group_size` is the number of processes in each server group. It should be an integer divisor of `io_size`.

The procedure to write data on disk with the IO server is the following. First an ostream is open by the compute processes. This ostream is the link between the compute and IO processes. It can be open only if the server is in the ready state, meaning that the server is started and is not writing data on disks. Therefore the procedure of opening an ostream returns a boolean to tell to the compute processes if the ostream is open. There is no wait-until-ready procedure: this needs to be hard